

If multiprocessing already allows multiple tasks using separate processes, why do we still need multithreading inside a single process?

Why Not Just Use Multiprocessin g?

- ◆ **Multiprocessing works, but it is “heavyweight”.**
- ◆ **Creating and managing processes requires more memory and time.**
- ◆ **Each process has its own separate address space.**
- ◆ **Communication is slow:** Processes communicate using IPC (pipes, sockets, etc.)
- ◆ **Higher overhead:** Context switching between processes is **costly**

Why Multithreading is Needed

Threads are lightweight

- Faster to create and manage than processes

Shared memory

- Threads share the same code and data
- Enables **fast communication**

Efficient for related tasks

- Ideal when tasks belong to the **same application**

When Multithreading Wins

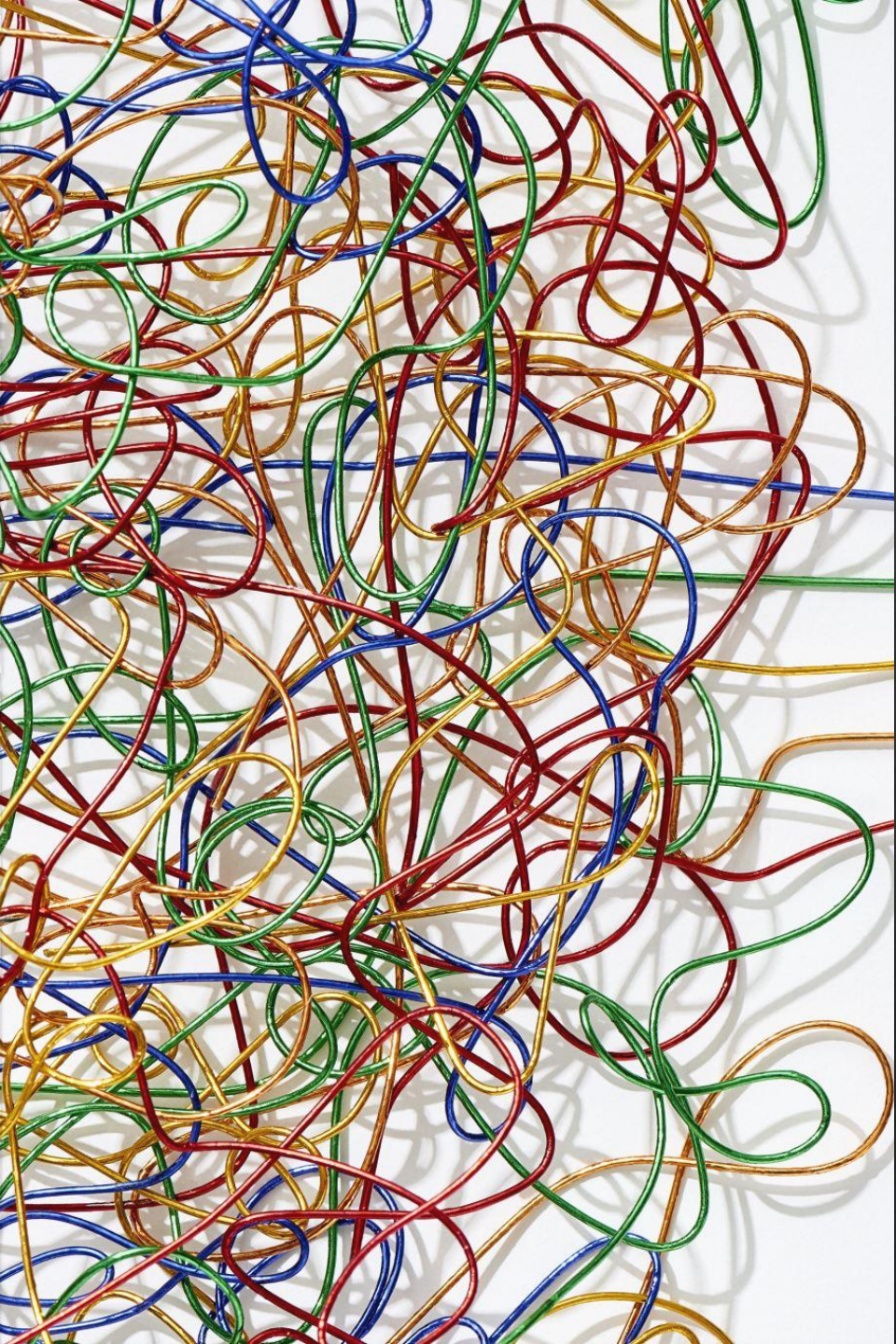
- ◆ When tasks need to:
 - Share data frequently
 - Run concurrently within the same program
 - Be more **responsive** (e.g., UI + background work)

Final Insight

- ◆ **Multiprocessing**
 - Better for isolation and safety
- ◆ **Multithreading**
 - Better for speed and efficient resource sharing
- ◆ That's why modern systems use both together, not one instead of the other.

What is a Thread?

- A **thread** is the **basic unit of CPU utilization**
- It represents a **single flow of execution**
- Each thread contains:
 - Thread ID
 - Program Counter (PC)
 - Register Set
 - Stack



What Threads Share

- ◆ Threads within the same process share:
 - ▣ Code section
 - ▣ Data section
 - ▣ OS resources (e.g., open files, signals)

Single-Threaded vs Multi-Threaded Process

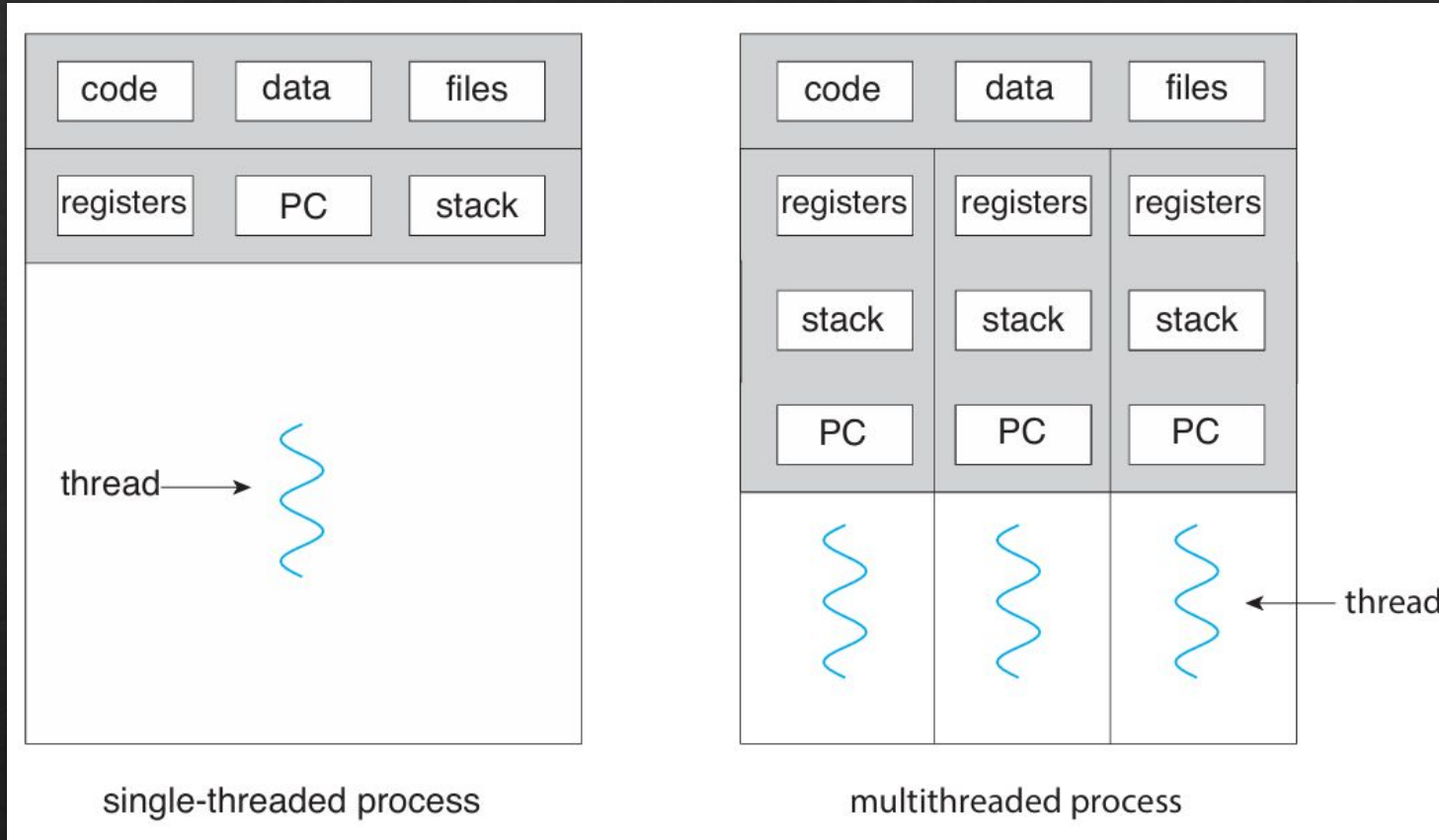
Single-threaded

- A traditional process has only **one thread of control**
- Can perform **one task at a time**

Multi-threaded

- A process with multiple threads can perform **multiple tasks simultaneously**
- Improves Performance and Responsiveness

Single-Threaded vs Multi-Threaded Process



Modern Computers

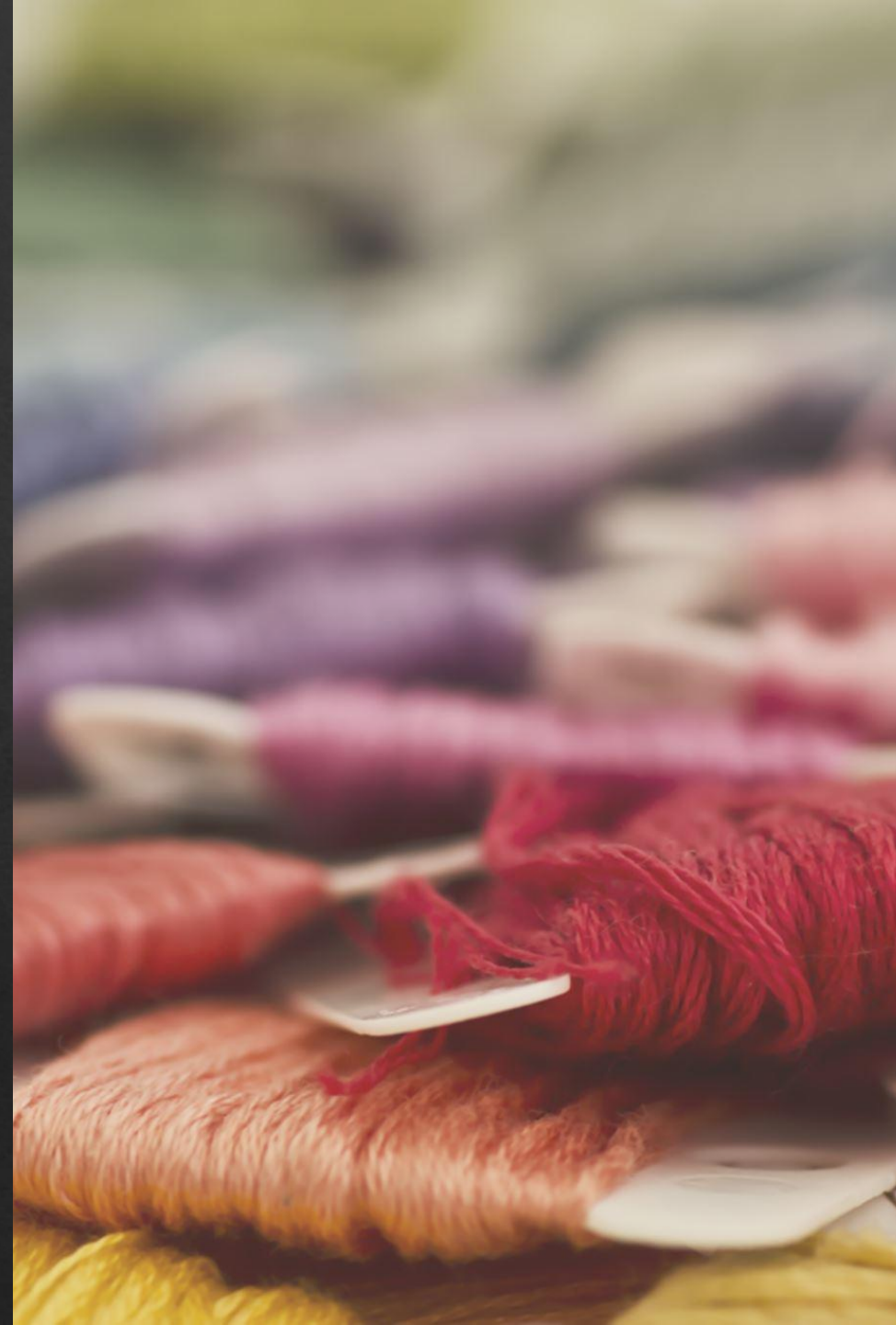
- Today's need:
 - Most modern software applications are **multithreaded**
 - Each application runs as a **process with multiple threads**
- Allows **multiple tasks to run simultaneously**
- Improves:
 - Performance
 - Responsiveness
 - User experience

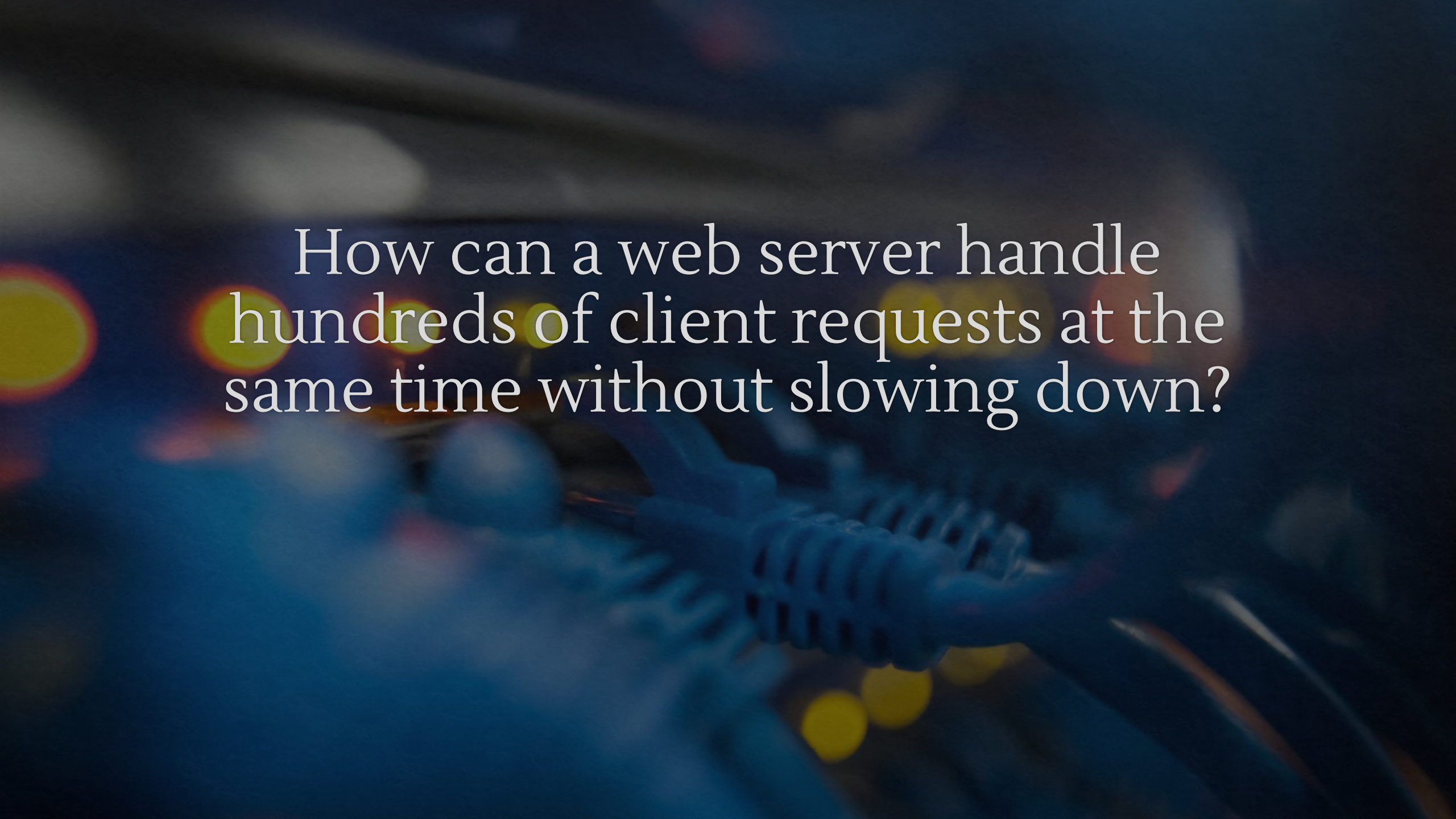
•



Example 1 – Image Processing App

- Creates thumbnails from images
- Uses:
 - One thread per image**
- Benefit:
 - Faster processing of multiple images at once



The background is a dark, out-of-focus image of a server room. It shows several blue network cables plugged into a patch panel. In the background, there are blurred yellow and orange lights, likely from server fans or indicators, creating a bokeh effect. The overall color palette is dark blue and black with some warm light accents.

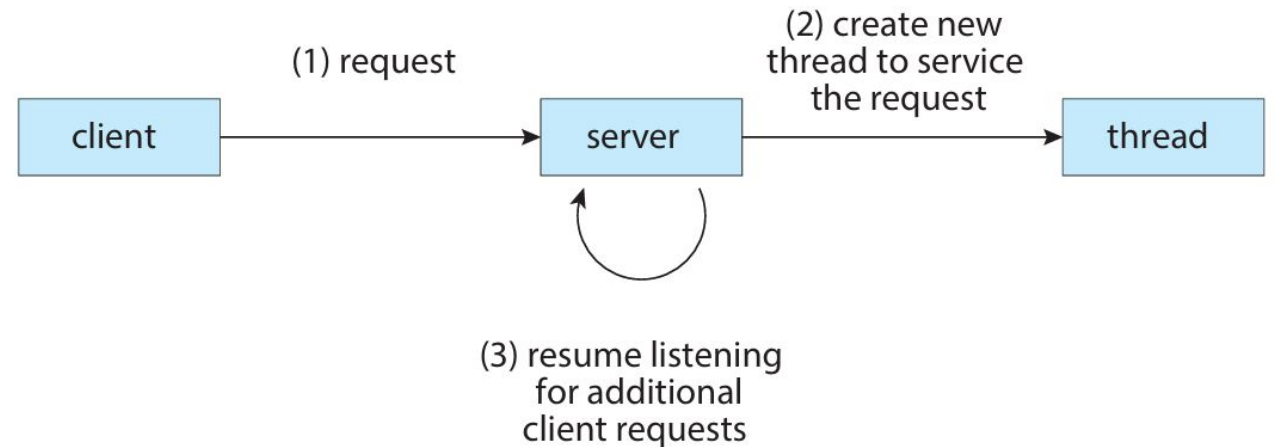
How can a web server handle
hundreds of client requests at the
same time without slowing down?

Traditional Process-Per -Request Approach

- Server runs as a **single process** that accepts client requests.
- For each request:
 - Creates a **new process** to handle it.
- **Drawbacks:**
 - Process creation is **time-consuming**
 - **Resource intensive** (memory + CPU)
 - Inefficient if all processes perform similar tasks

Multithreaded Server Approach

- Single server process contains **multiple threads**
- One thread **listens for incoming requests**
- For each request:
 - A **new thread** is created to service it
 - Listening thread **continues accepting new requests**



Why Multithreading in Server–Client ?

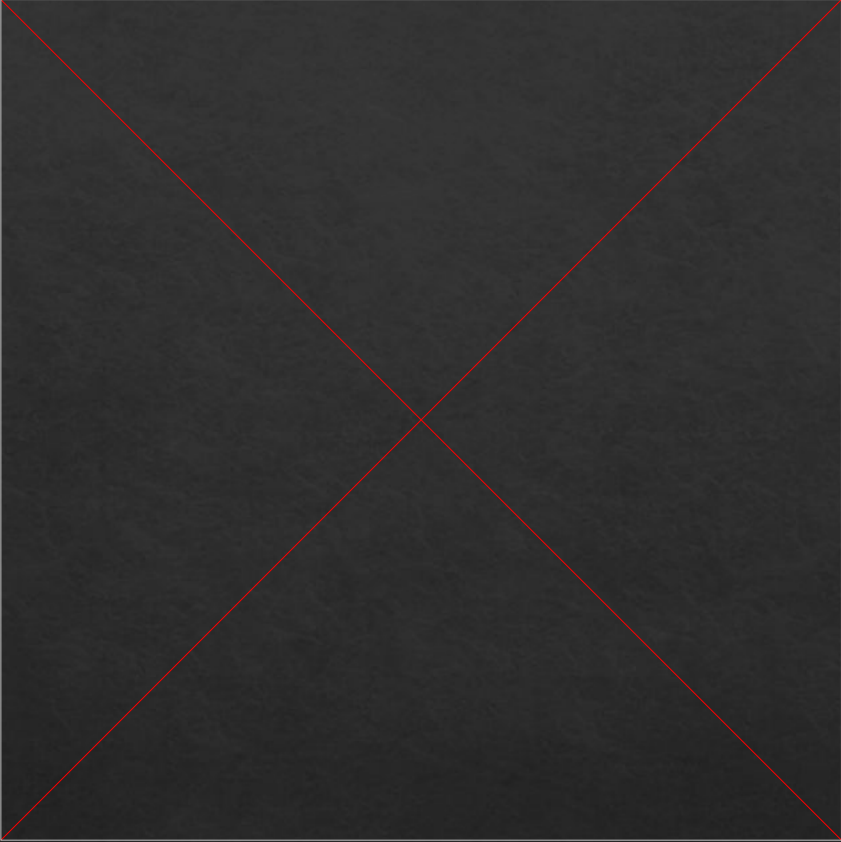
- **Using Multiprocessing:**

- One process per client request
- High overhead (memory + creation time)
- Slower when requests are large in number

- **Using Multithreading (within one process):**

- One thread per client request
- Faster request handling
- Lower resource usage (shared memory)
- Efficient communication (no heavy IPC)

Real Scenario (e.g., 50 Client Requests)



- **Recoverable failure:**

- A thread encountering a minor error can terminate safely.
- Other threads continue servicing clients.

- **Critical failure:**

- A thread crashing (e.g., segmentation fault) can **affect shared memory**.
- May cause the **entire server process to stop**, terminating all threads.

How Real Systems Handle This



Use **thread pools**



Implement
exception handling



Use **hybrid models**
(multi-process +
multithreading)



Restart failed
threads/processes
automatically

Benefits of Multithreaded Programming

Responsiveness

- Allows applications to continue running even if part is blocked or performing a lengthy operation
- Example: Time-consuming tasks run in a **separate thread** , keeping the UI responsive

Resource Sharing

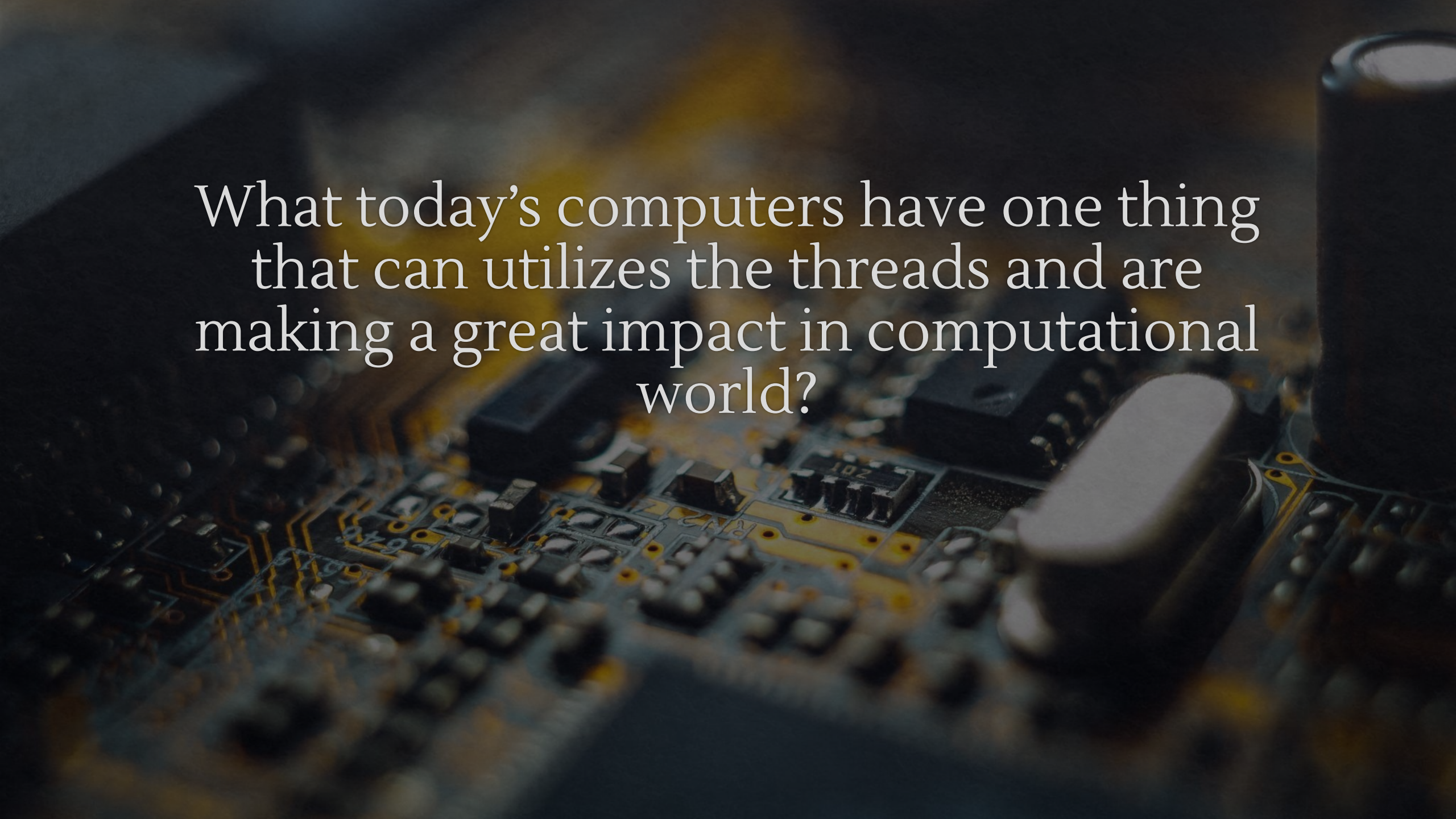
- Threads **share memory and resources** of their parent process by default
- Enables multiple threads to work within the **same address space** without complex IPC

Economy

- Thread creation and context switching is **less costly** than creating processes
- Saves **time and memory**

Scalability

- Threads can run **in parallel on multiple processor cores**
- Single-threaded processes are limited to **one processor**



What today's computers have one thing
that can utilizes the threads and are
making a great impact in computational
world?

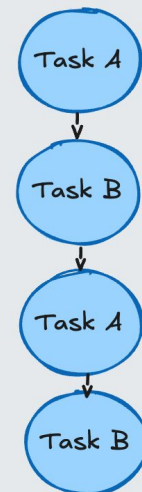
Introduction to Multicore

- Modern CPUs often have **multiple cores** (physical processing units).
- Each core can execute instructions (in threads) independently.
- Benefits:
 - Higher throughput
 - Better multitasking

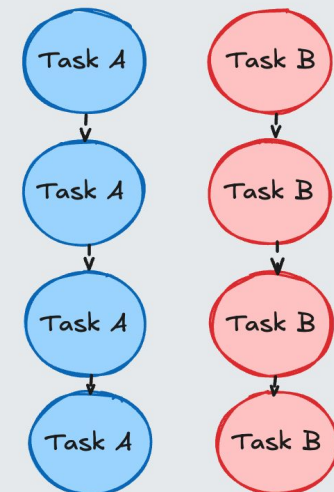
Concurrency vs Parallelism

Concept	Meaning	Example
Concurrency	Multiple tasks make progress simultaneously , but not necessarily at the same instant	Single-core CPU switching between threads
Parallelism	Multiple tasks execute at the same time on different cores	Multi-core CPU running multiple threads simultaneously

Concurrency

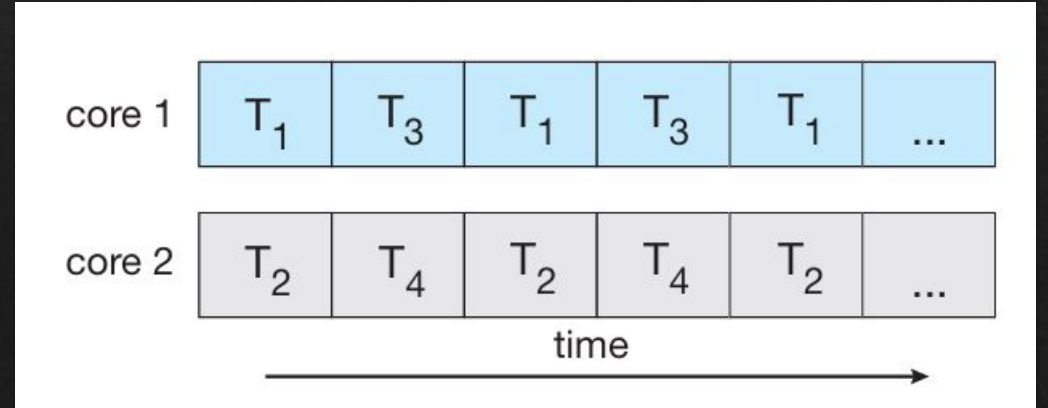


Parallelism



Concurrency + Parallelism

- Modern multicore systems often use both:
 - **Concurrency** → managing multiple tasks together
 - **Parallelism** → executing tasks at the same time on different cores
- Modern applications rely on both concurrency and parallelism to fully utilize multicore processors



Key Insight

Concurrency : Focus on structure and managing multiple tasks

Parallelism : Focus on **speedup** by running tasks simultaneously

Multithreading + multicore → enables both **concurrent** and **parallel execution**

When using multiple cores, should we divide the work itself or divide the data being processed?

Types of Parallelism

- **Task Parallelism**

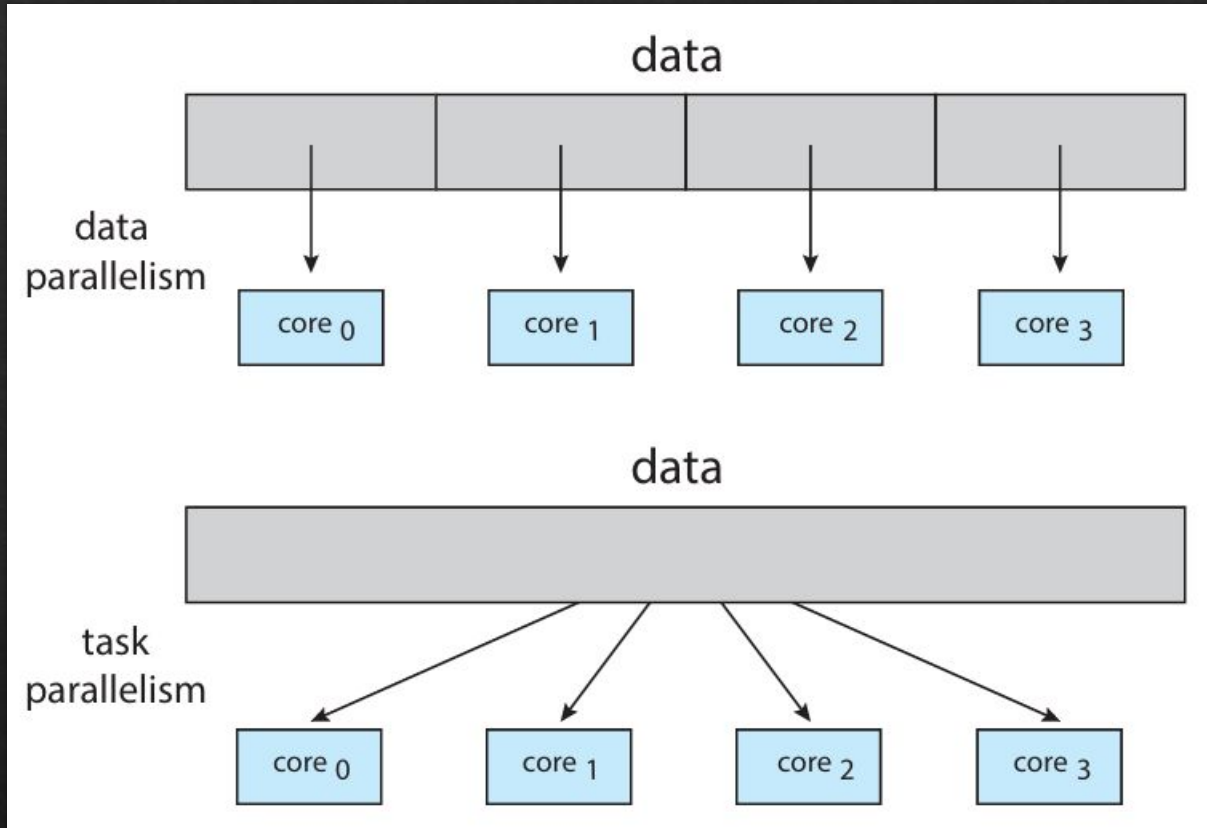
- Different tasks run in parallel
- Example: One thread handles user input, another handles file saving

- **Data Parallelism**

- Same task runs on different parts of data in parallel
- Example: Splitting a large image into sections and processing each section separately



Illustration

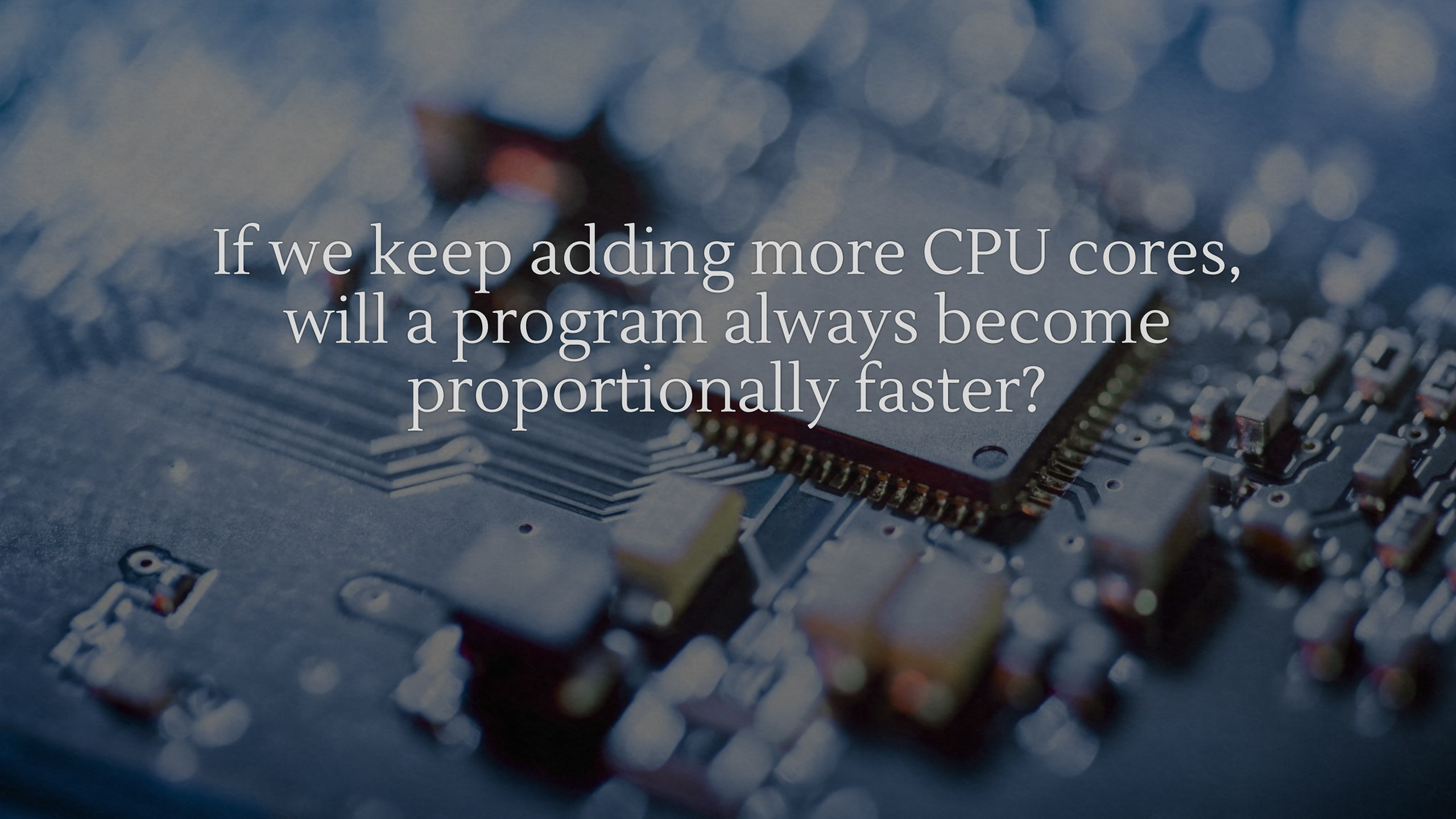


Parallelism in AI Training

Task Parallelism	Data Parallelism
Different tasks run in parallel	Same training task runs on different parts of the dataset
Example: One thread loads data, another trains the model, another evaluates accuracy	Example: Dataset is split into batches and processed simultaneously on multiple GPUs/cores
Focuses on dividing work by activity	Focuses on dividing work by data

Summary

Task Parallelism	Data Parallelism
Different tasks run in parallel	Same task runs on different data
Divides work by activity	Divides work by data
Example: Browser functions	Example: Image processing



If we keep adding more CPU cores,
will a program always become
proportionally faster?

Amdahl's Law

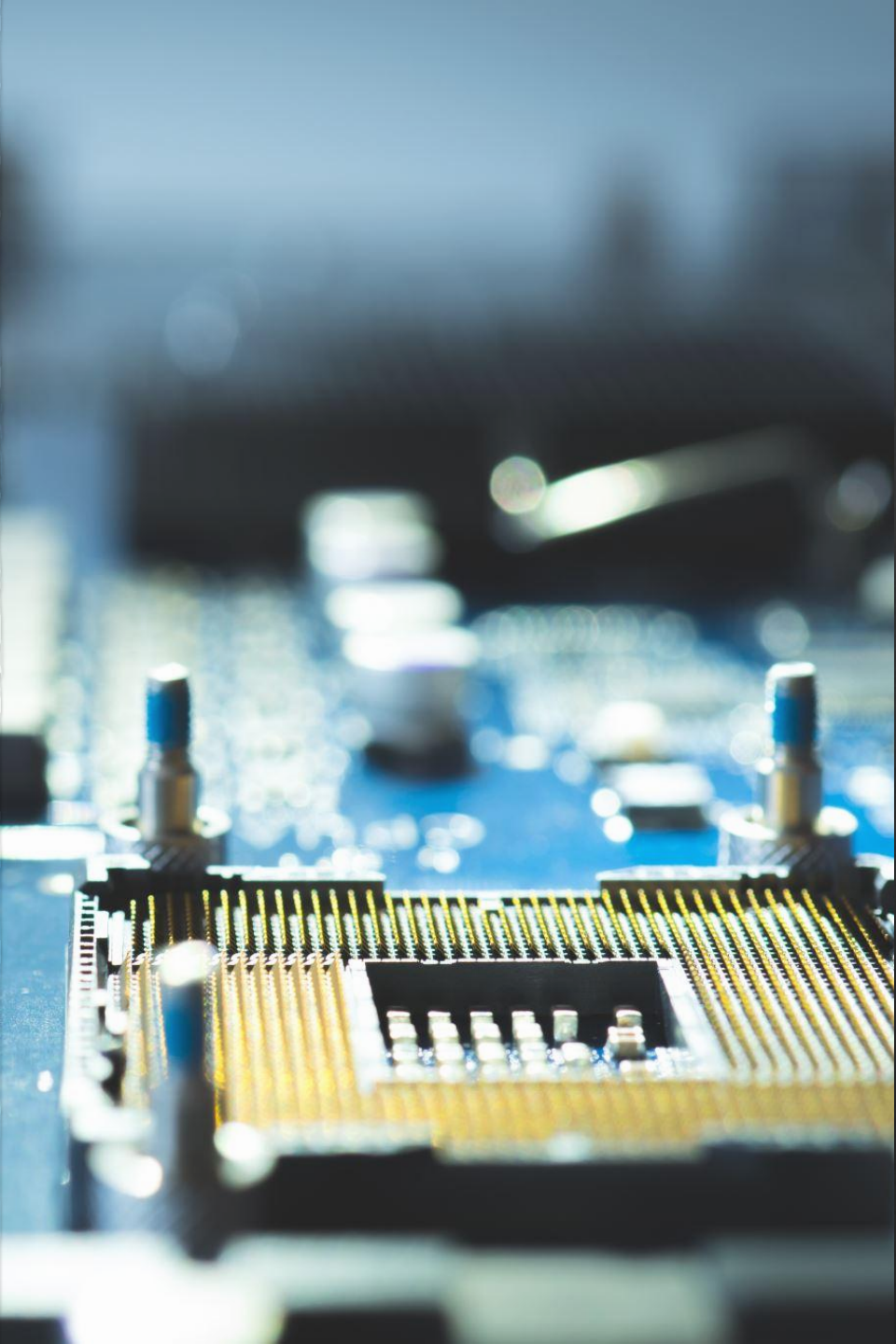
Amdahl's Law : Explains the **potential speedup of a program** when using multiple processors compared to a single processor.

Performance gain is **limited by the portion of the program that must run sequentially** .

Even with many processors, the sequential part becomes a **bottleneck** .

Amdhal's Law

- ◆ Imagine a program has **two parts** :
 - A part that can run in **parallel** (many processors can work on it).
 - A part that is **sequential** (only one processor can do it).
- ◆ **Sequential part = the limit.** No matter how many processors you add, that sequential portion will always take the same time.



Example

- ♦ A program takes **10 hours** .
- ♦ **8 hours** can run in parallel, **2 hours** must run sequentially.
- ♦ With 100 processors → the 8 hours shrink almost to zero, but the **2 hours remain**.
- ♦ So, the program **still takes at least 2 hours** s.
- ♦ Adding more processors helps only up to a point.
- ♦ The part that can't be parallelized sets a **hard limit** on speedup.

Formula

- ◆ 1. When f is small, the use of parallel processors has little effect.
- ◆ 2. As N approaches infinity, speedup is bound by $1 / (1-f)$, for using more processors.

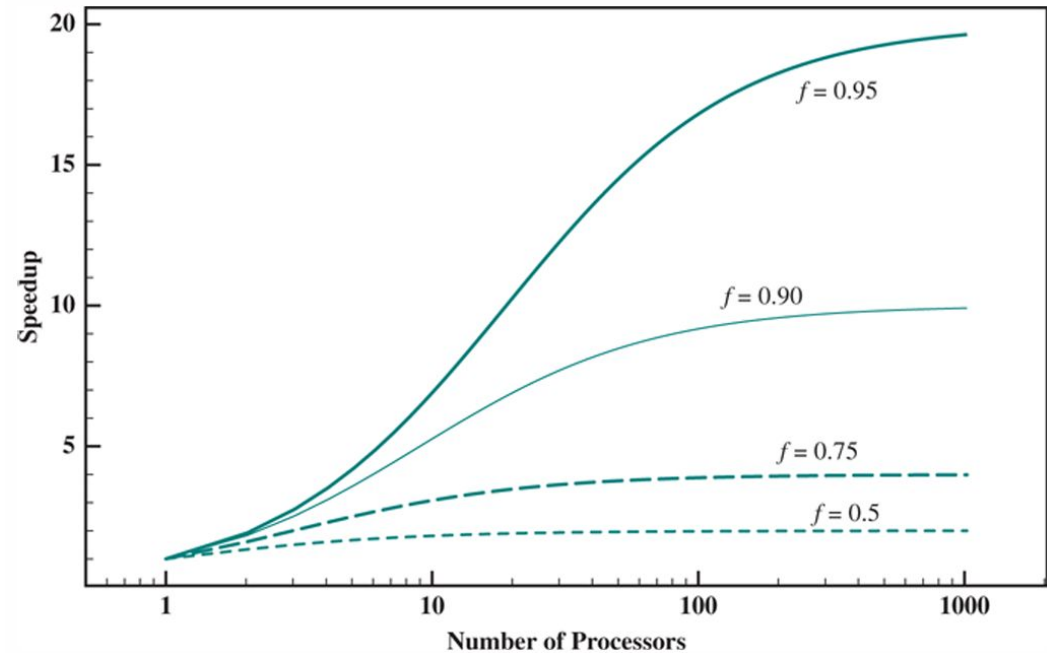
$$S = \frac{1}{(1 - F) + \frac{F}{N}}$$

Where:

- S = Speedup (overall performance gain)
- F = Fraction of program that can be parallelized ($0 \leq F \leq 1$)
- N = Number of processors

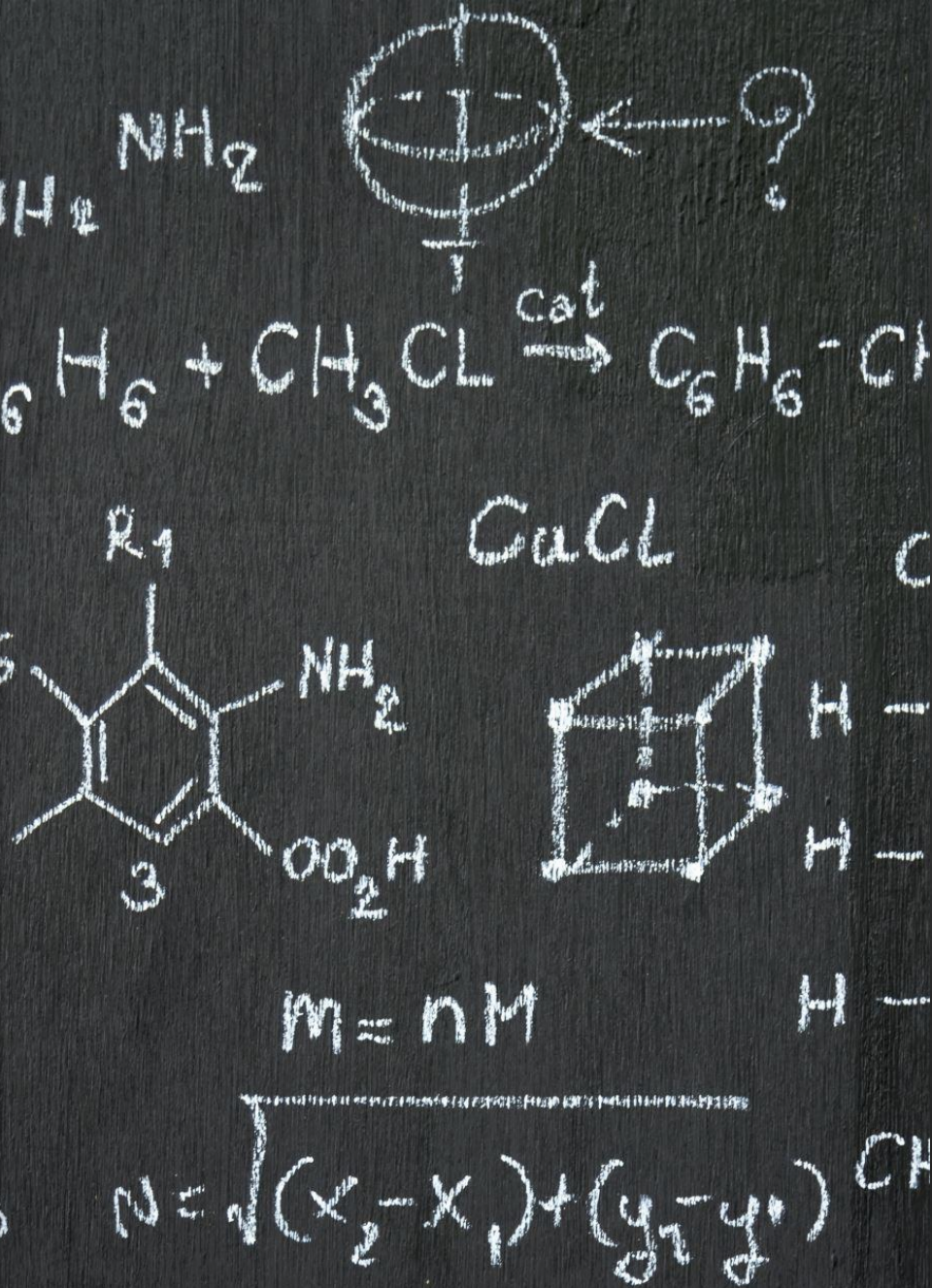
Amdahl's Law and Number of Processors

- ◆ The speed-up of a program from parallelization is limited by how much of the program can be parallelized.
- ◆ For example, if 95% of the program can be parallelized, the theoretical maximum speedup using parallel computing would be 20× as shown in the diagram, no matter how many processors are used.
- ◆ If 90% of the program can be parallelized, the theoretical maximum speed-up using parallel computing would be 10×



Practical Examples

- ◆ **1. Operating System (OS)**
- ◆ **Boot sequence** : The OS must load the kernel before drivers, and drivers before applications.
 - You can't run them in parallel because each step depends on the previous one.
- ◆ **2. Graphics (GPU rendering)**
- ◆ **Frame rendering pipeline** :
 - You must first **calculate geometry** before applying **textures** .
 - Finally, you **display** the frame.
 - Each stage depends on the previous result, so parts must be sequential even if others run in parallel.
- ◆ **3. Robotics**
- ◆ **Sensor → Processing → Action loop** :
 - Robot must **read sensor data** → then **process/decide** → then **send motor commands** .
 - You can parallelize sensor fusion or path planning, but the **decision-to-action flow is sequential** , otherwise the robot might act on incomplete info.
- ◆ That's why Amdahl's Law matters: even if you have **100 processors** , the sequential parts (like boot, rendering order, or sensor-to-action flow) will always limit the maximum speedup.



Example Calculation

- ◆ Application:
 - ▣ 25% serial (1-F)
 - ▣ 75% parallel (F)
- ◆ With 2 cores (N):
 - ▣ Speedup (S) ≈ 1.6 times
- ◆ With 4 cores:
 - ▣ Speedup (S) ≈ 2.28 times
- ◆ Adding more cores improves performance, but not linearly

Key Takeaway

More cores do not always mean unlimited performance gain

The serial portion of a program limits overall speedup

To fully benefit from multicore systems, applications should minimize serial work

User Threads vs Kernel Threads

User Threads:

- Managed in user space
- Do not require direct kernel support
- Faster to create and manage

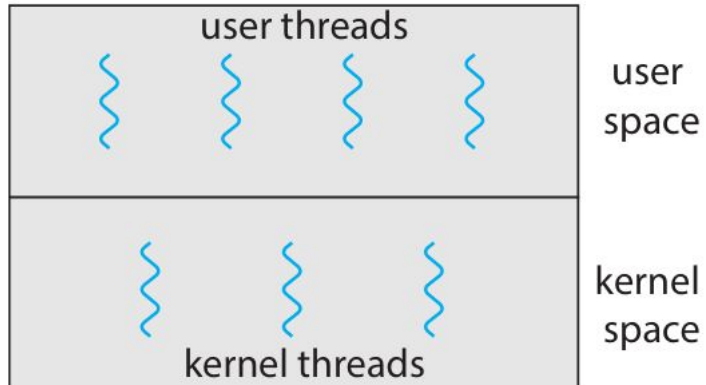
Kernel Threads:

- Managed directly by the operating system
- Supported by the kernel
- Used by most modern operating systems

Modern OSs such as Windows, Linux, and macOS mainly support kernel threads

Which is Better?

- There is no single “best” type
- Choice depends on the application requirements



User Threads

Faster to create and switch

Low overhead

If one thread blocks, entire process may block

Limited ability to use multiple CPU cores

Managed by thread libraries

Kernel Threads

Slower due to kernel involvement

Higher overhead

One blocked thread does not stop others

Can run truly in parallel on multicore systems

Managed by operating system

When to Use User Threads

- Suitable when:
 - Very fast thread creation is needed
 - Application performs lightweight tasks
 - Limited OS interaction is required
- Example:
 - Simple simulations
 - Lightweight user-level scheduling libraries



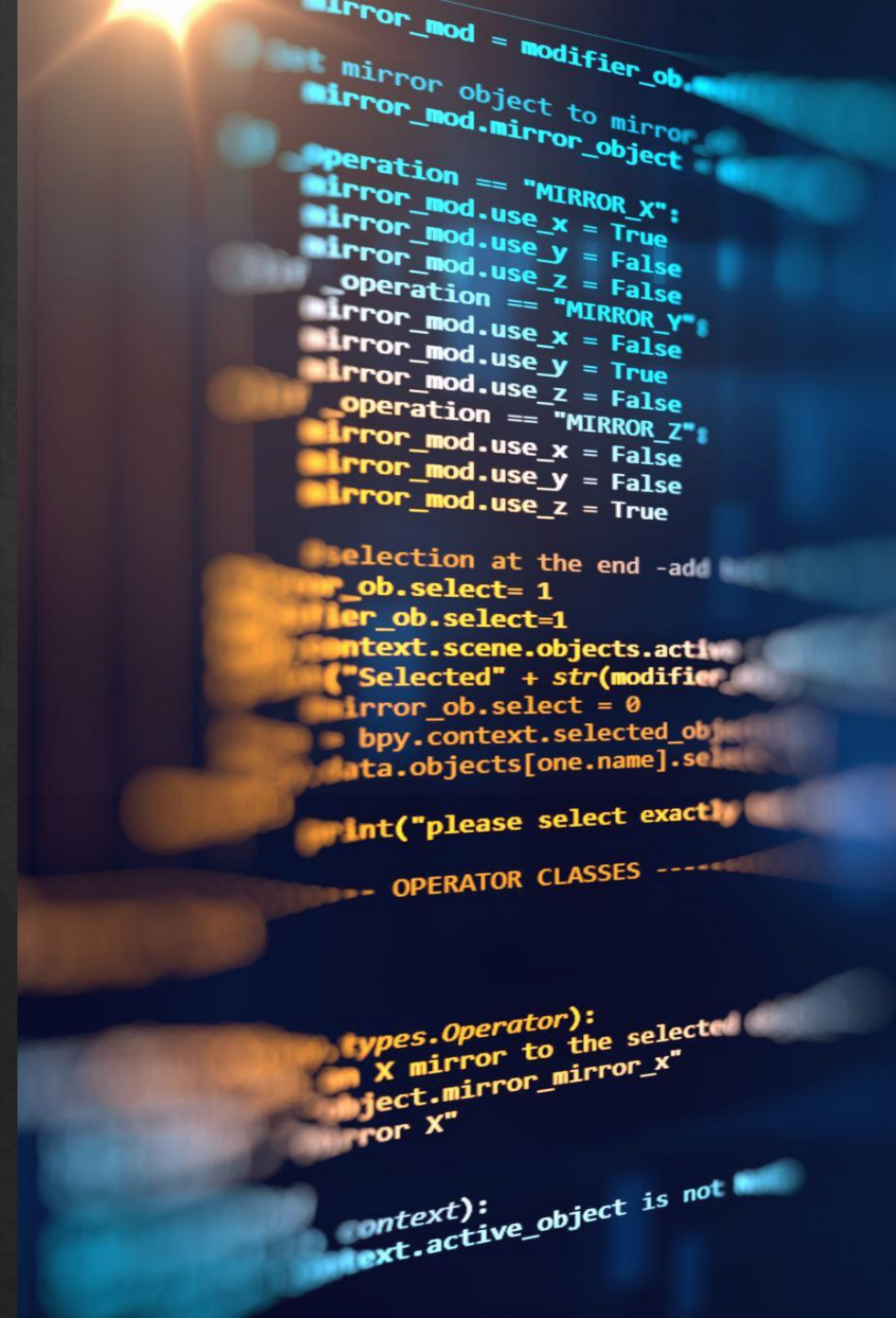
When to Use Kernel Threads

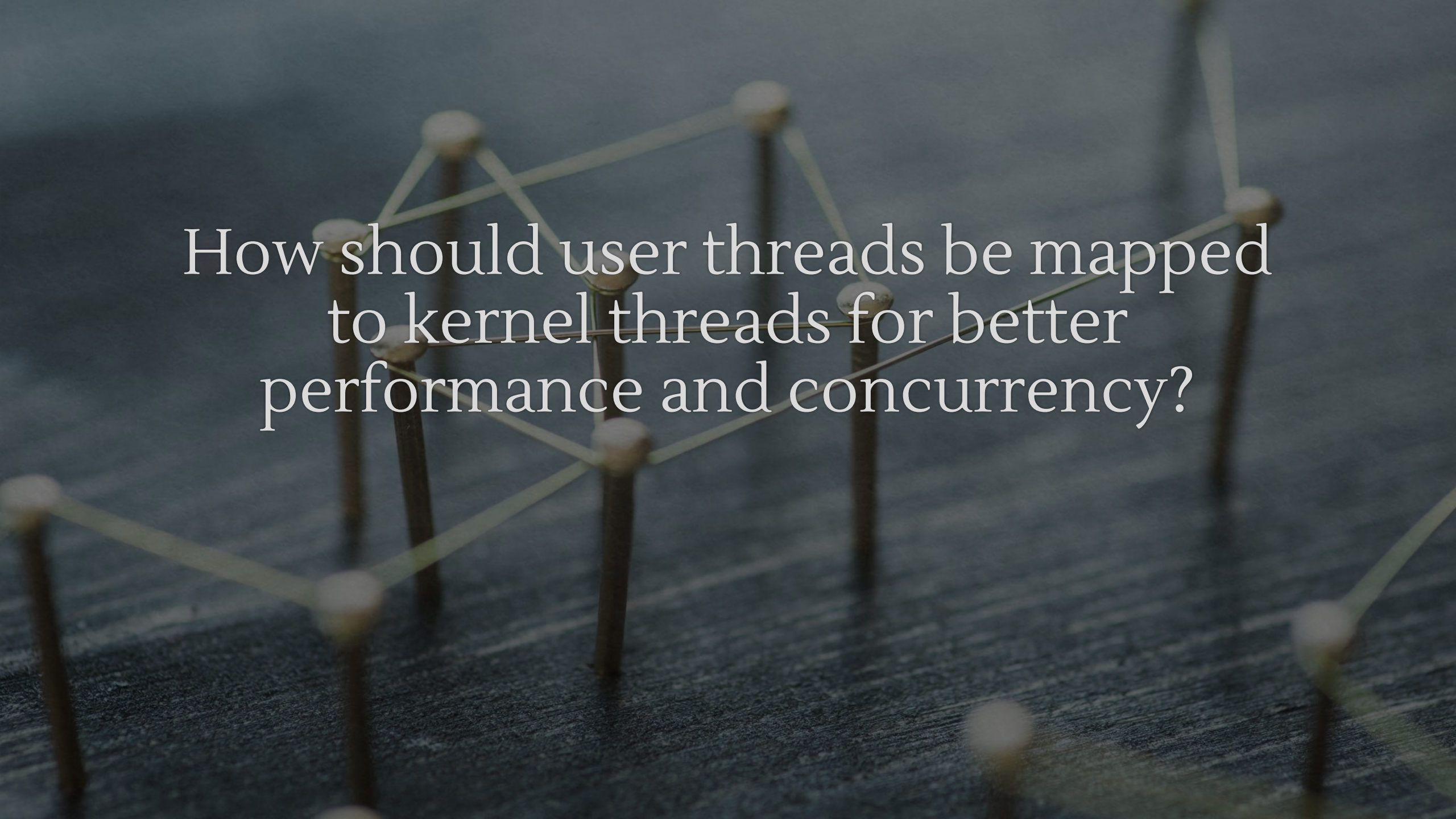
- Suitable when:

- True parallelism is required
- Application performs I/O operations frequently
- Reliability and blocking isolation are important

- Example:

- Web servers
- Browsers
- Database systems
- Modern AI and parallel computing application

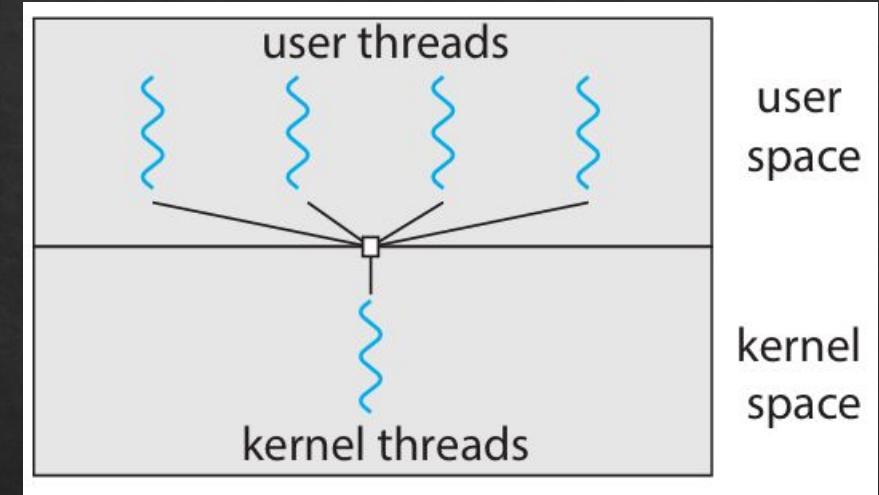


A network diagram with nodes and connections on a dark background. The nodes are represented by small, light-colored spheres, and the connections are thin, light-colored lines. The diagram is composed of several interconnected triangles and other geometric shapes, suggesting a complex network structure. The background is dark and textured, with some faint, larger-scale geometric patterns visible.

How should user threads be mapped
to kernel threads for better
performance and concurrency?

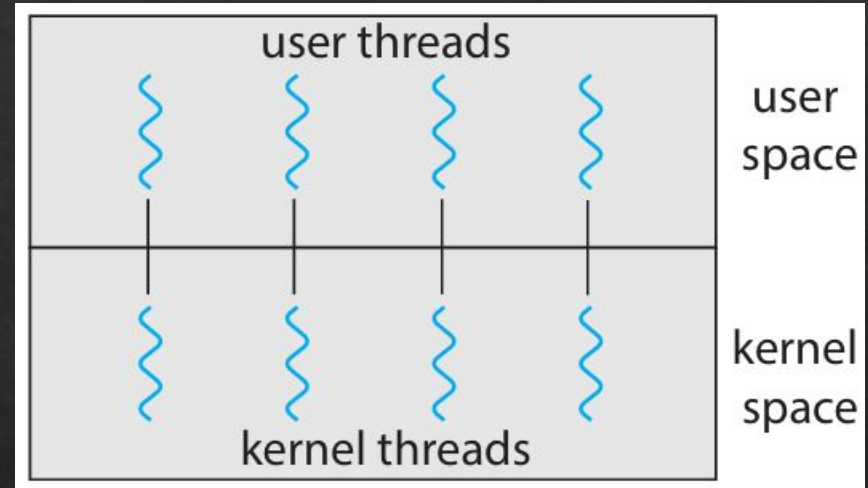
Many-to-One Model

- Many user threads map to one kernel thread
- Thread management is done in user space
- ✓ Fast and efficient
- ! If one thread blocks, entire process blocks
- ! Cannot achieve true parallelism on multicore systems



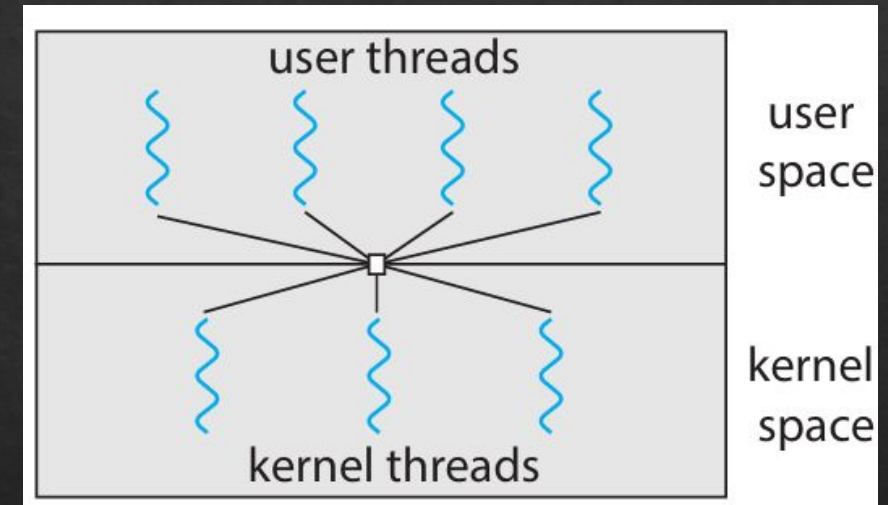
One-to-One Model

- Each user thread maps to one kernel thread
- ✓ Better concurrency
- ✓ One blocked thread does not stop others
- ✓ Supports true parallelism on multicore processors
- ! Creating many kernel threads increases overhead
- Used by modern operating systems such as Linux and Windows



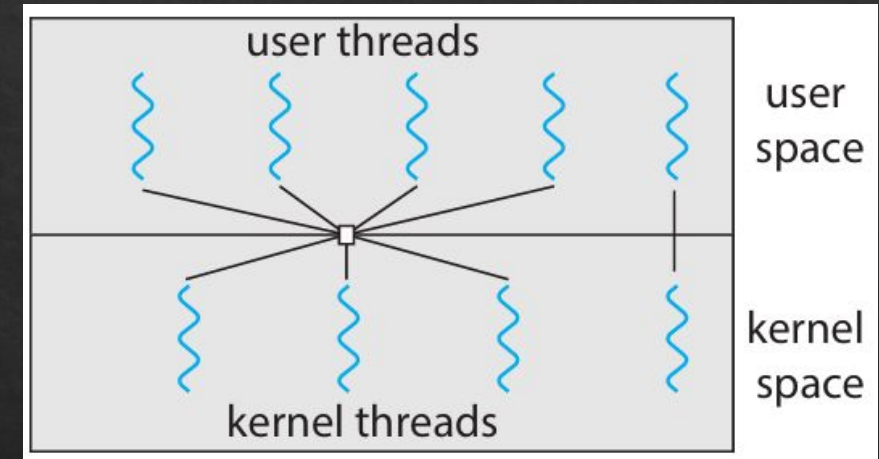
Many-to-Many Model

- Many user threads map to a smaller or equal number of kernel threads
- ✓ Allows many user threads without creating too many kernel threads
- ✓ Supports parallelism
- ✓ If one thread blocks, others can continue
- More flexible than many-to-one and one-to-one



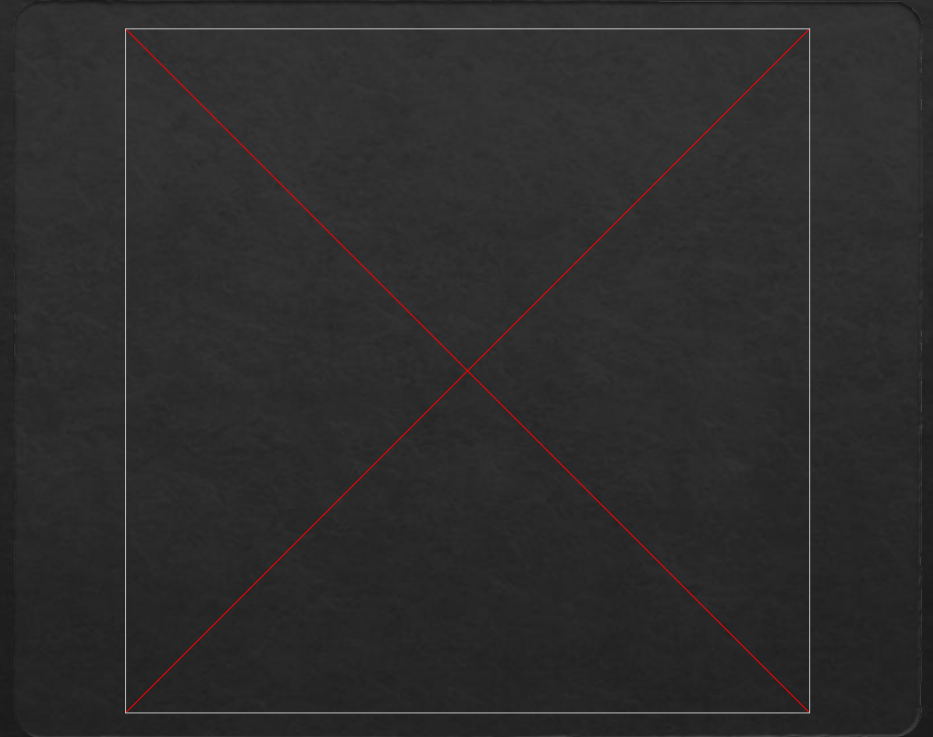
Two-Level Model

- Variation of many-to-many model
- Allows some user threads to be directly bound to specific kernel threads
- Provides both:
 - Flexibility of many-to-many
 - Dedicated kernel thread support when needed



Which Model is Most Common Today?

- Many-to-one is rarely used because it cannot utilize multicore systems
- Many-to-many is flexible but difficult to implement
- One-to-one is most common in modern operating systems because it provides:
 - Better concurrency
 - Parallelism
 - Simpler implementation



Summary of Multithreading Models

Model	Mapping	Advantages	Disadvantages
Many-to-One	Many user threads → One kernel thread	Fast and low overhead	No parallelism, one blocked thread blocks all
One-to-One	One user thread → One kernel thread	Better concurrency and true parallelism	Higher overhead due to many kernel threads
Many-to-Many	Many user threads → Many kernel threads	Flexible, supports concurrency and parallelism	More complex to implement
Two-Level	Many-to-Many with optional direct binding	Combines flexibility with dedicated kernel threads	Complex design